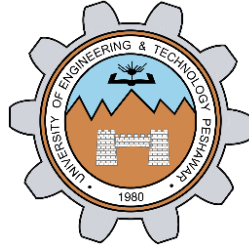


Project Report



Spring 2024

CSE-308L Digital System Design Lab

Submitted by:

Ali Asghar(21PWCSE2059)

Shahzad Bangash(21PWCSE1980)

Suleman Shah(21PWCSE1983)

Abu Bakar(21PWCSE2004)

Class Section: **C**

Submitted to:

Engr. Shahzada Fahim Jan

Date:

26th June 2024

**Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar**

Handwritten Image Recognition on FPGA

Abstract

This report presents the implementation of a simple neural network using High-Level Synthesis (HLS) targeting an FPGA. The network consists of two dense layers with ReLU and softmax activations. The design is optimized for performance using HLS directives.

Contents

1	Introduction	2
1.1	Neural Network	2
1.2	Why neural networks?	2
1.3	Field Programmable Gate Arrays (FPGAs)	2
1.4	Why FPGAs?	2
1.5	FPGA Design Workflow	3
1.6	Coding in FPGA	3
1.7	Hardware Synthesis in FPGA	4
1.8	High-Level Synthesis (HLS)	4
2	Neural Network Implementation	6
2.1	Model Training	6
2.1.1	Model Training Data	6
2.2	Structure of Model	7
2.3	Exponential Approximation	7
2.4	ReLU Activation	8
2.5	Softmax Activation	9
2.6	Dense Layers	9
2.7	Top-level Function	10
3	Implementation	12
3.1	Design Flow	12
3.2	Synthesis and Implementation	12
4	Results	13
4.1	Testing Methodology	13
4.2	Performance Metrics	13
5	Conclusion	14

Chapter 1

Introduction

1.1 Neural Network

A neural network is a method in artificial intelligence that teaches computers to process data in a way that is inspired by the human brain. It is a type of machine learning process, called deep learning, that uses interconnected nodes or neurons in a layered structure that resembles the human brain. It creates an adaptive system that computers use to learn from their mistakes and improve continuously. Thus, artificial neural networks attempt to solve complicated problems, like summarizing documents or recognizing faces, with greater accuracy[1].

1.2 Why neural networks?

Neural networks can help computers make intelligent decisions with limited human assistance. This is because they can learn and model the relationships between input and output data that are nonlinear and complex. They are widely used in making smart systems nowadays.

1.3 Field Programmable Gate Arrays (FPGAs)

FPGAs are integrated circuits that can be configured by the user after manufacturing. They consist of an array of programmable logic blocks and a hierarchy of reconfigurable interconnects, allowing the creation of custom hardware circuits.

1.4 Why FPGAs?

Field Programmable Gate Arrays (FPGAs) offer several advantages for implementing neural networks:

- **Parallelism:** FPGAs can exploit the inherent parallelism in neural network computations, allowing for concurrent execution of multiple operations. This leads to significant speedup compared to sequential processing on general-purpose processors.
- **Customization:** FPGAs can be customized to implement specific neural network architectures and operations, optimizing the hardware for the target application. This flexibility allows designers to balance performance, power consumption, and resource utilization.
- **Low Latency:** FPGAs provide deterministic and low-latency processing, which is critical for real-time applications such as autonomous driving, robotics, and financial trading.
- **Energy Efficiency:** FPGAs can be more energy-efficient than CPUs and GPUs for certain tasks, as they can be tailored to perform only the necessary computations with minimal overhead.
- **Reconfigurability:** FPGAs can be reconfigured to implement different neural network models or updated designs without changing the hardware. This reconfigurability makes FPGAs suitable for research and development environments where algorithms are constantly evolving.

1.5 FPGA Design Workflow

The workflow of an FPGA Based Design[2] is given in the figure 1.1. Figure 1.1 depicts a typical workflow for designing, verifying, and implementing a chip architecture. It begins with the "Concept" phase, where the idea and specifications for the chip are defined. This is followed by the "Design" phase, which includes developing the chip architecture and writing the hardware description language (HDL) code. The design is then subjected to "Verification" through simulation and testing to ensure it meets the specified requirements and functions correctly. If any issues are detected, the design is revised. Once verified, the design moves to the "Implementation" phase, where it undergoes synthesis to convert the HDL code into a gate-level netlist, and then place and route to arrange and connect the gates physically. This structured approach ensures that each step is thoroughly checked and corrected before progressing, thereby reducing errors and ensuring successful chip fabrication.

1.6 Coding in FPGA

Coding for FPGAs (Field Programmable Gate Arrays) involves writing hardware description language (HDL) code to define the behavior and structure of digital circuits. The two most commonly used HDLs are VHDL (VHSIC Hardware Description Language) and Verilog. Coding in FPGA involves several key considerations and steps to ensure efficient and functional hardware design.

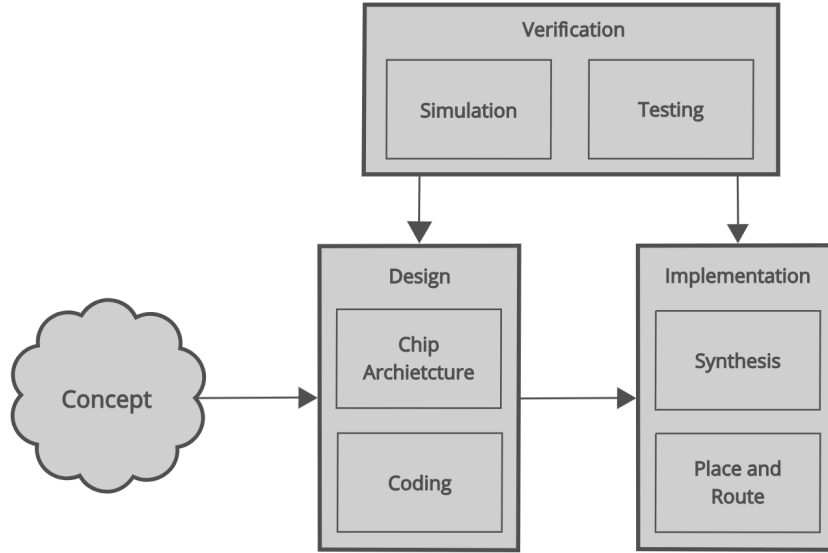


Figure 1.1: FPGA Based Design Workflow

Coding for FPGAs requires a deep understanding of both hardware and software principles. Efficient FPGA coding can lead to highly optimized designs that exploit the parallelism and reconfigurability of FPGAs, making them ideal for applications requiring high performance, low latency, and energy efficiency. Proper coding practices, thorough verification, and effective use of synthesis tools are essential to successful FPGA design.

1.7 Hardware Synthesis in FPGA

During the FPGA synthesis process[3], a high description design or an HDL design is converted into a gate level representation or a logic component. This means that FPGA code provided in high level language such as VHDL or Verilog is converted into logic gates using a synthesis tool as shown in figure 1.2.

1.8 High-Level Synthesis (HLS)

HLS is a method of transforming a high-level algorithmic description of a circuit into a register-transfer level (RTL) design. HLS tools can automatically generate hardware descriptions from C, C++, or SystemC code, significantly reducing the design time. Its general workflow can be seen in figure 1.3.

FPGA Synthesis



Figure 1.2: FPGA Synthesis Workflow

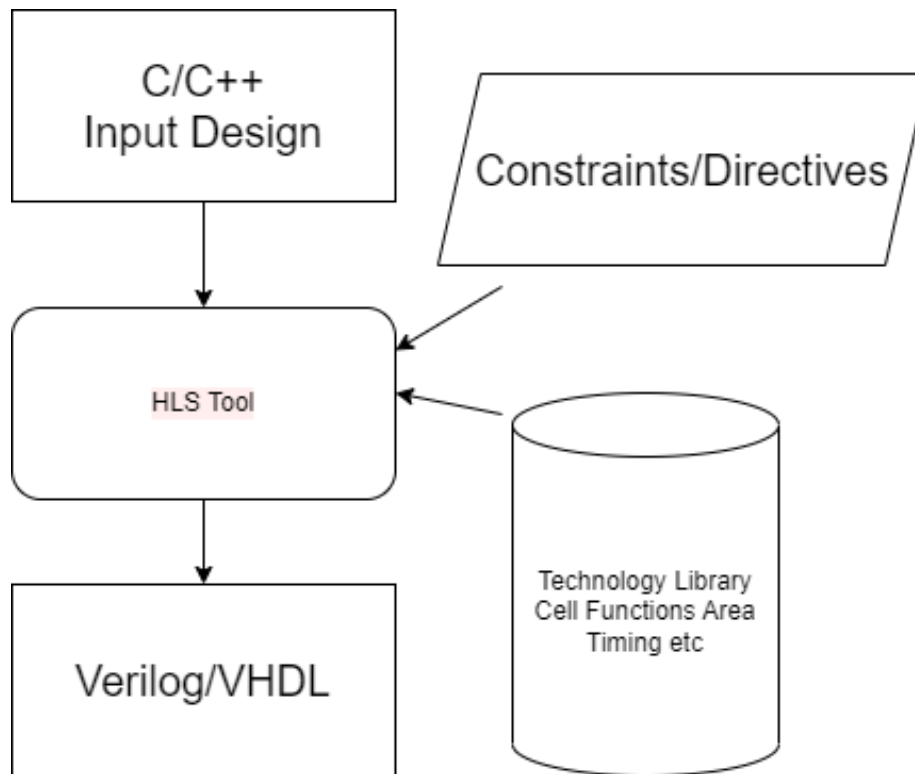


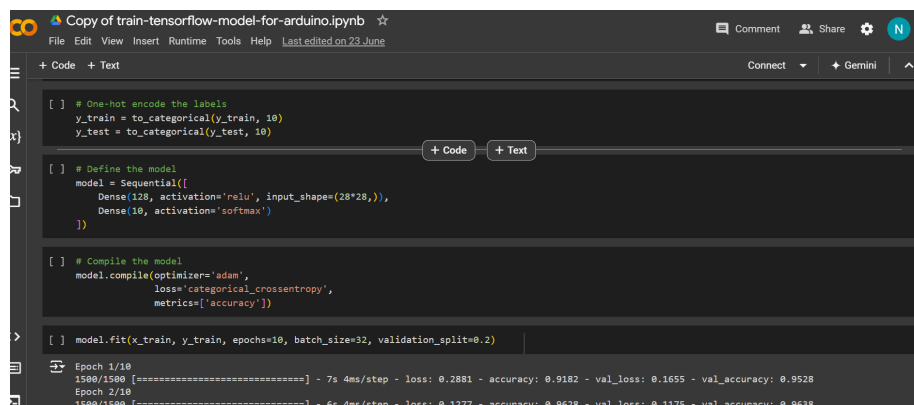
Figure 1.3: HLS Synthesis Workflow

Chapter 2

Neural Network Implementation

2.1 Model Training

First step is to train a machine learning model using tools like Jupyter Notebook, Google Colab etc. In this project, we have used Google Colab for training a tensor flow model for handwritten digit recognition as shown in figure 2.1.



```
[ ] # One-hot encode the labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

[ ] # Define the model
model = Sequential([
    Dense(128, activation='relu', input_shape=(28*28,)),
    Dense(10, activation='softmax')
])

[ ] # Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

[ ] model.fit(x_train, y_train, epochs=10, batch_size=32, validation_split=0.2)

Epoch 1/10
1500/1500 [=====] - 7s 4ms/step - loss: 0.2881 - accuracy: 0.9182 - val_loss: 0.1655 - val_accuracy: 0.9528
Epoch 2/10
1500/1500 [=====] - 6s 4ms/step - loss: 0.1277 - accuracy: 0.9628 - val_loss: 0.1175 - val_accuracy: 0.9618
```

Figure 2.1: Model Training in Google Colab

2.1.1 Model Training Data

For training, we have used MNIST dataset. The MNIST (Modified National Institute of Standards and Technology) dataset is a benchmark dataset widely used in the field of machine learning and computer vision, specifically for handwritten digit recognition. It consists of images of handwritten digits from 0 to 9.

9 and serves as a standard for evaluating and comparing the performance of various algorithms and models. It consist of images as shown in figure 2.3.

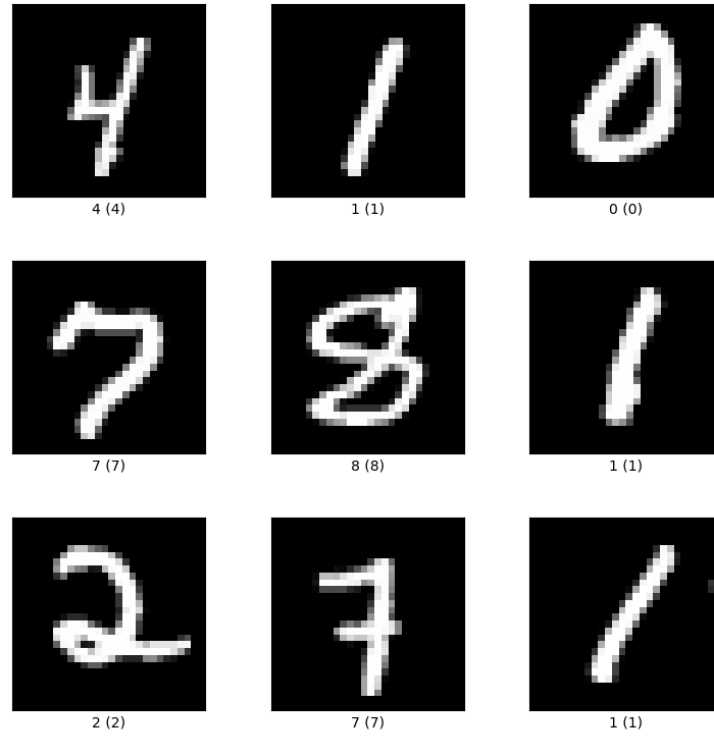


Figure 2.2: MNIST Dataset Sample

2.2 Structure of Model

The model consists of two dense (fully connected) layers. The first layer has 128 hidden units, and the second layer has 10 output units corresponding to 10 possible classes. The network architecture is as follows:

- Input layer: 784 units (28x28 pixels)
- Dense layer 1: 128 units
- ReLU activation

- Dense layer 2: 10 units
- Softmax activation

2.3 Exponential Approximation

The exponential function is approximated using a truncated Taylor series for computational efficiency:

```
data_t exp_approx(data_t x) {
    const int num_terms = 10;
    data_t result = 1.0;
    data_t term = 1.0;

    for (int i = 1; i < num_terms; ++i) {
        term *= x / i;
        result += term;
    }
    return result;
}
```

2.4 ReLU Activation

The ReLU activation function is applied element-wise to the output of the first dense layer:

```
void relu(data_t input[HIDDEN_SIZE], data_t output[HIDDEN_SIZE]) {
    for (int i = 0; i < HIDDEN_SIZE; i++) {
        output[i] = input[i] > 0 ? input[i] : 0;
    }
}
```

2.5 Softmax Activation

The softmax function normalizes the output of the second dense layer to a probability distribution:

```
void softmax(data_t input[OUTPUT_SIZE], data_t output[OUTPUT_SIZE]) {
    data_t max_val = input[0];
    for (int i = 1; i < OUTPUT_SIZE; i++) {
        if (input[i] > max_val) {
            max_val = input[i];
        }
    }
}
```

```

    data_t sum = 0;
    for (int i = 0; i < OUTPUT_SIZE; i++) {
        output[i] = exp_approx(input[i] - max_val);
        sum += output[i];
    }

    for (int i = 0; i < OUTPUT_SIZE; i++) {
        output[i] /= sum;
    }
}

```

2.6 Dense Layers

The dense layers are implemented as follows:

```

void dense_1(data_t input[INPUT_SIZE], data_t output[HIDDEN_SIZE], const data_t
    for (int i = 0; i < HIDDEN_SIZE; i++) {
        data_t sum = bias[i];
        for (int j = 0; j < INPUT_SIZE; j++) {
            sum += input[j] * weights[i][j];
        }
        output[i] = sum;
    }
}

void dense_2(data_t input[HIDDEN_SIZE], data_t output[OUTPUT_SIZE], const data_t
    for (int i = 0; i < OUTPUT_SIZE; i++) {
        data_t sum = bias[i];
        for (int j = 0; j < HIDDEN_SIZE; j++) {
            sum += input[j] * weights[i][j];
        }
        output[i] = sum;
    }
}

```

2.7 Top-level Function

The top-level function integrates the layers and activation functions:

```

void neural_network(data_t input[INPUT_SIZE], data_t output[OUTPUT_SIZE],
    const data_t dense_1_weights[INPUT_SIZE][HIDDEN_SIZE], const
    const data_t dense_2_weights[HIDDEN_SIZE][OUTPUT_SIZE], cons

#pragma HLS INTERFACE m_axi port=input offset=slave bundle=gmem
#pragma HLS INTERFACE m_axi port=output offset=slave bundle=gmem

```

```

#pragma HLS INTERFACE m_axi port=dense_1_weights offset=slave bundle=gmem
#pragma HLS INTERFACE m_axi port=dense_1_bias offset=slave bundle=gmem
#pragma HLS INTERFACE m_axi port=dense_2_weights offset=slave bundle=gmem
#pragma HLS INTERFACE m_axi port=dense_2_bias offset=slave bundle=gmem
#pragma HLS INTERFACE s_axilite port=return bundle=control

data_t dense_1_out[HIDDEN_SIZE];
data_t relu_out[HIDDEN_SIZE];
data_t dense_2_out[OUTPUT_SIZE];

// Layer 1: Dense
dense_1(input, dense_1_out, dense_1_weights, dense_1_bias);

// Layer 1: ReLU Activation
relu(dense_1_out, relu_out);

// Layer 2: Dense
dense_2(relu_out, dense_2_out, dense_2_weights, dense_2_bias);

// Layer 2: Softmax Activation
softmax(dense_2_out, output);
}

```

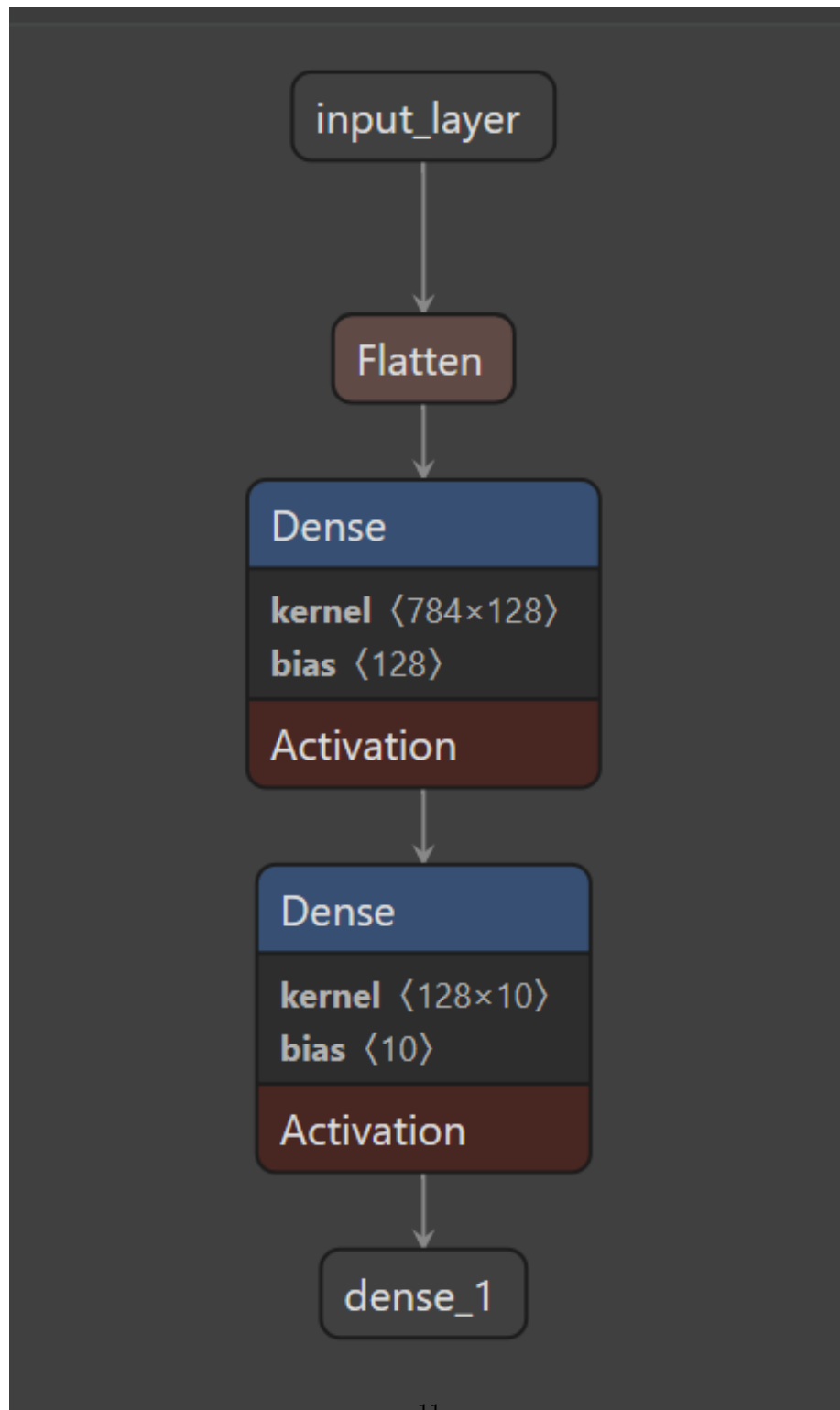


Figure 2.3: Structure of ML Model in Netron

Chapter 3

Implementation

This chapter describes the implementation details of the neural network on the FPGA. It includes the design flow, synthesis, and implementation steps.

3.1 Design Flow

The design flow involves the following steps:

- Writing the high-level description in C++
- Applying HLS directives for optimization
- Synthesizing the design to generate RTL code
- Implementing the design on the FPGA
- Validating the design using test data

3.2 Synthesis and Implementation

Using the HLS toolchain, the C++ code is synthesized into RTL, followed by place and route on the FPGA. The HLS directives such as `#pragma HLS INTERFACE` are used to optimize memory access and parallelism.

Chapter 4

Results

The performance and accuracy of the implemented neural network are evaluated. This chapter includes the testing methodology, performance metrics, and comparison with other implementations.

4.1 Testing Methodology

The neural network is tested using a dataset of images. The inputs are processed through the network, and the outputs are compared with the expected results to evaluate accuracy.

4.2 Performance Metrics

The performance of the network is measured in terms of latency, throughput, and resource utilization on the FPGA. The results are summarized in the following table:

Metric	Value	Description
Latency	xxx ns	Time taken for a single inference
Throughput	xxx images/sec	Number of images processed per second
Resource Utilization	xxx LUTs, xxx FFs, xxx BRAMs	FPGA resource usage

Table 4.1: Performance Metrics

Chapter 5

Conclusion

This report demonstrated the implementation of a neural network on an FPGA using HLS. The design achieved efficient performance by leveraging hardware parallelism and custom approximations for mathematical functions. Future work includes optimizing the design further and extending it to more complex networks.

References

- [1] <https://aws.amazon.com/what-is/neural-network>
- [2] <https://fpgatutorial.com/introduction-to-fpga-development/>
- [3] <https://hardwarebee.com/understanding-fpga-logic-synthesis/>